PROJEKTPRÄSENTATION

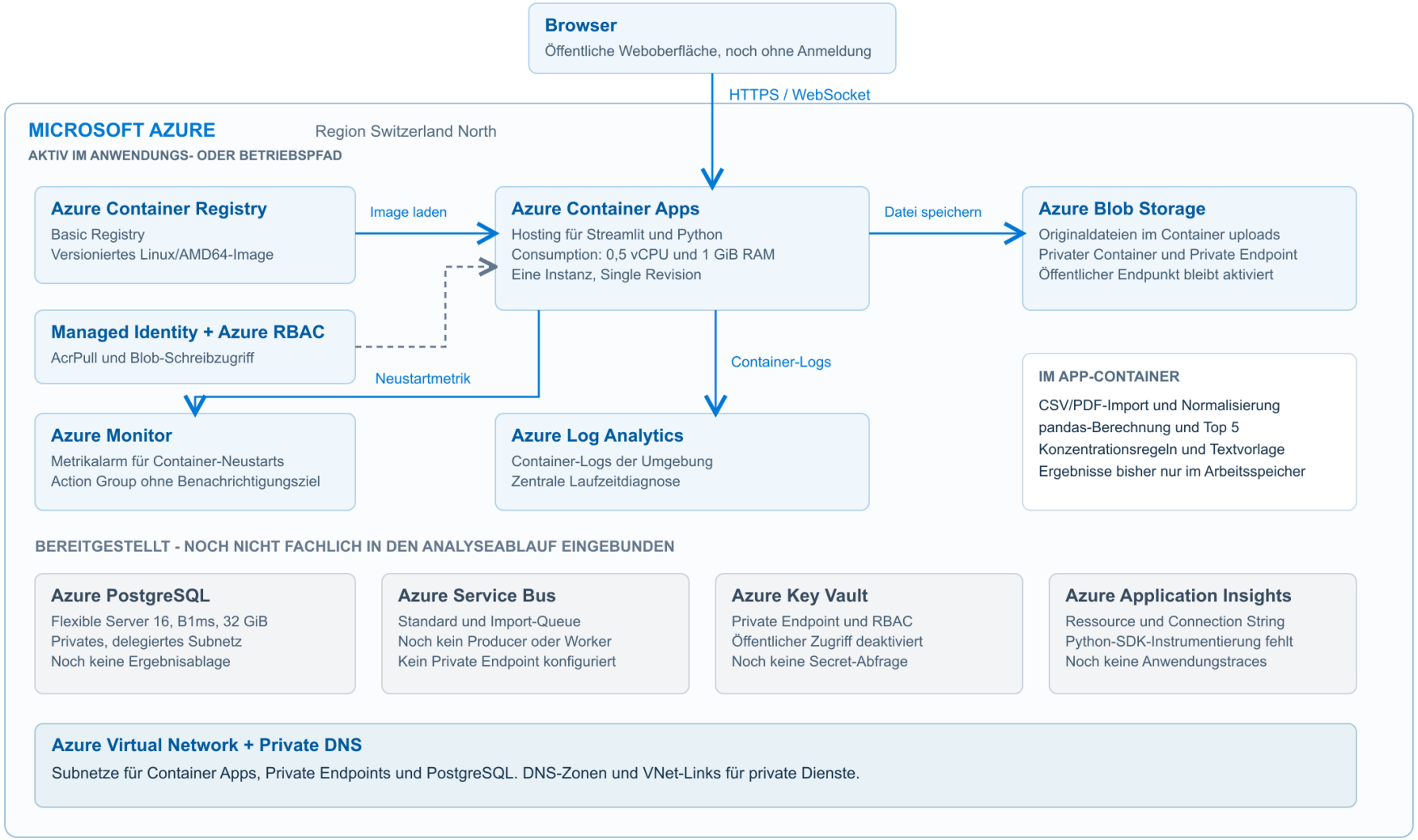
Example Azure Depot PDF

Depotaufstellungen analysieren
Python-Anwendung auf Azure Container Apps

Technischer Stand: 27. Oktober 2026

Example Azure Depot PDF

Azure-Architektur: eingesetzte Dienste und ihr tatsächlicher Implementierungsstand



Bereitstellung: Terraform und ACR Build. GitHub Actions prüft Docker-Build und CLI, ohne automatisches Azure-Deployment.

Azure App Service ist nicht eingesetzt. Die Anwendung läuft auf Azure Container Apps. Stand: 27. Oktober 2026.

Der Analyseablauf

01 Upload	CSV oder PDF im Browser auswählen. Die App speichert die Datei bei aktiviertem Azure-Zugriff in Blob Storage.
02 Import	Der Importer normalisiert Spalten und Zahlen. PDF-Import nutzt erkennbare Tabellen aus pdfplumber.
03 Berechnung	pandas berechnet Gesamtwert, Positionsgewichte und die fünf größten Positionen.
04 Ergebnis	Streamlit zeigt Tabellen und regelbasierte Konzentrationshinweise. Eine Textvorlage fasst die Ergebnisse zusammen.

Die Anwendung im Betrieb

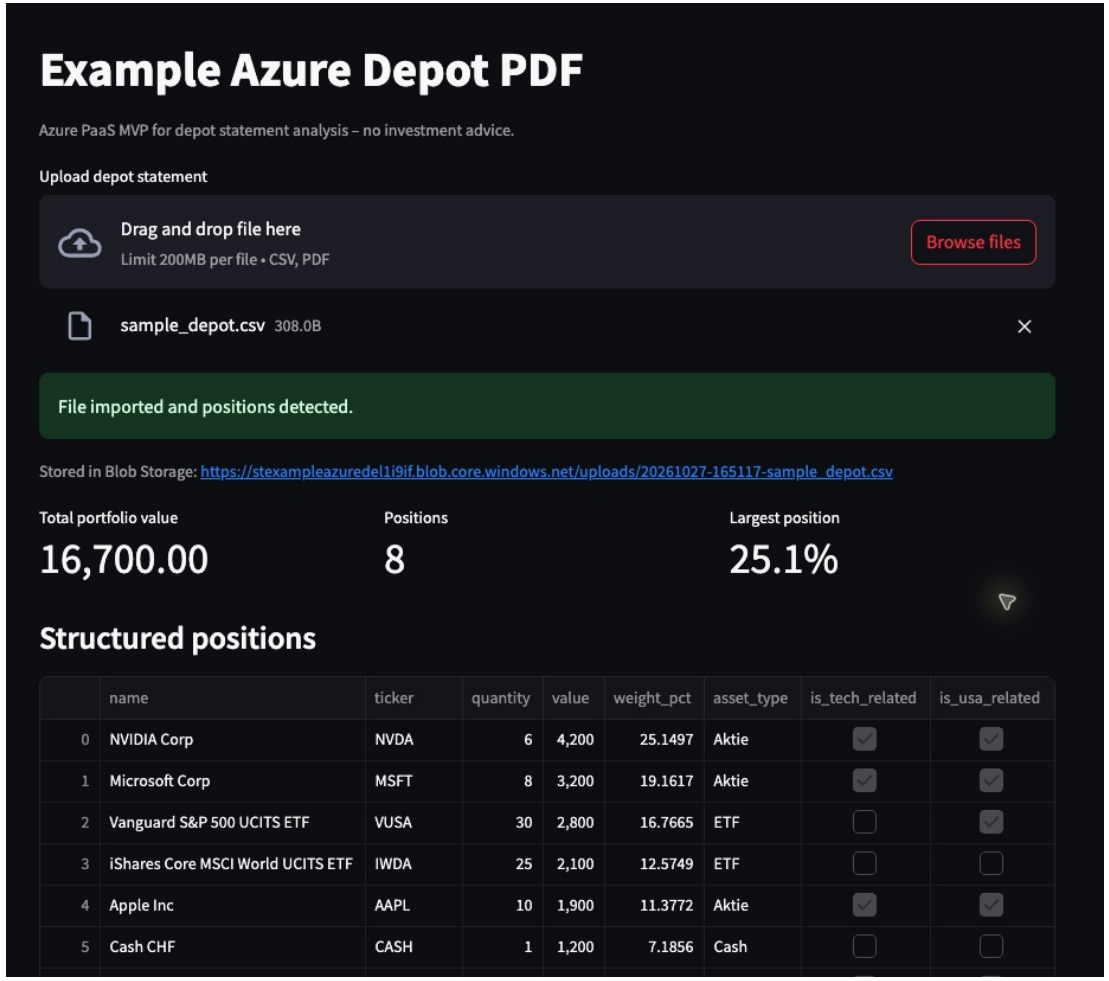
8

Positionen in der Beispieldatei

16.700,00

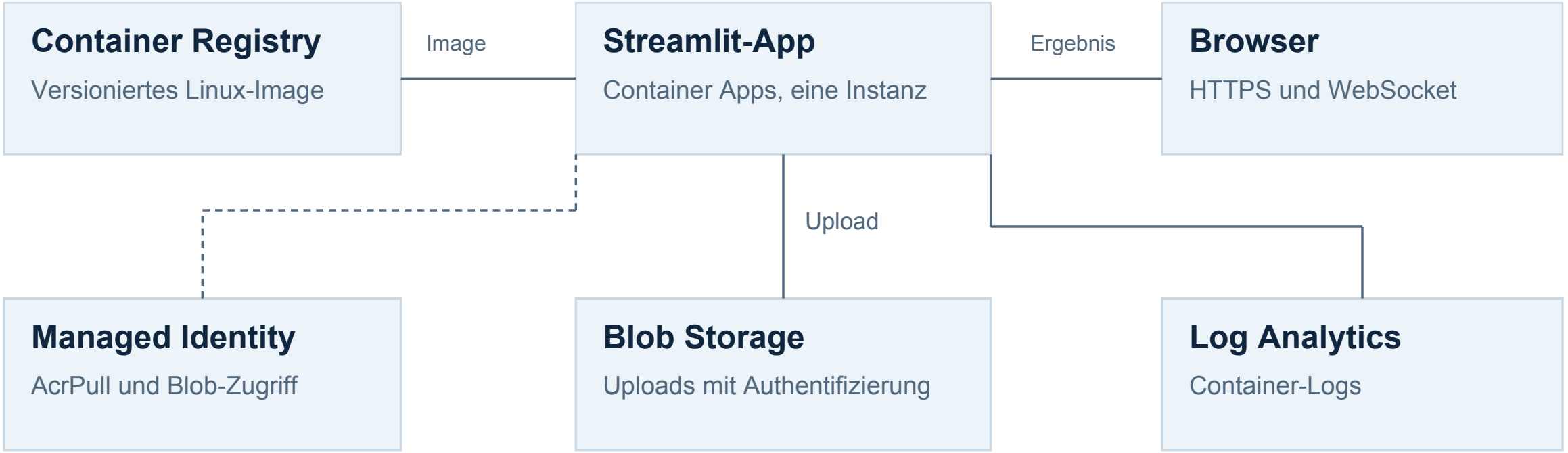
Berechneter Gesamtwert
25,1 % größtes Positionsgewicht

Browser-Test: Upload, Blob-Speicherung und Tabellenanzeige erfolgreich.



	name	ticker	quantity	value	weight_pct	asset_type	is_tech_related	is_usa_related
0	NVIDIA Corp	NVDA	6	4,200	25.1497	Aktie	☑	☑
1	Microsoft Corp	MSFT	8	3,200	19.1617	Aktie	☑	☑
2	Vanguard S&P 500 UCITS ETF	VUSA	30	2,800	16.7665	ETF	☐	☑
3	iShares Core MSCI World UCITS ETF	IWDA	25	2,100	12.5749	ETF	☐	☐
4	Apple Inc	AAPL	10	1,900	11.3772	Aktie	☑	☑
5	Cash CHF	CASH	1	1,200	7.1856	Cash	☐	☐

Aktive Verbindungen



Import, Kennzahlen und Textzusammenfassung laufen im Streamlit-Prozess. PostgreSQL und Service Bus gehören noch nicht zum aktiven Analysepfad.

Netzwerkstruktur



Der Browser erreicht die App über den öffentlichen HTTPS-Ingress. Die App nutzt den privaten Storage-Pfad. PostgreSQL und Key Vault bleiben privat erreichbar.

Datenhaltung und Fehlerverhalten

Originaldateien	Blob Storage bewahrt konfigurierte Uploads über Container-Neustarts hinweg auf. Ein automatischer Löschjob für aktive Dateien fehlt.
Analyseergebnisse	Positionen, Kennzahlen und Zusammenfassung liegen im Arbeitsspeicher. Eine dauerhafte Ergebnisablage ist noch nicht implementiert.
Storage-Fehler	Wenn die konfigurierte Speicherung scheitert, zeigt die App den Fehler an. Der Analyseaufruf erfolgt dann nicht.
Importfehler	Die Speicherung erfolgt vor der Validierung. Eine nicht auswertbare Datei kann bereits im Blob-Container liegen bleiben.

Bereitgestellte Erweiterungsdienste

PostgreSQL 16	B1ms mit 32 GiB Speicher im privaten Subnetz. Der Analysecode speichert noch keine Ergebnisse in der Datenbank.
Service Bus Standard	Namespace und Import-Queue sind vorhanden. Producer und Hintergrund-Worker fehlen noch.
Key Vault	Private Endpoint und RBAC sind konfiguriert. Die App fragt aktuell keine Secrets ab.

Application Insights ist bereitgestellt. Für Anwendungstraces benötigt der Python-Code noch eine SDK-Instrumentierung.

Netzwerk und Zugriffsschutz

Anwendungsidentität	Die App wählt ihre zugewiesene Managed Identity über AZURE_CLIENT_ID. Zugriffsschlüssel stehen nicht im Anwendungscode.
Private Anbindung	PostgreSQL und Key Vault haben keinen öffentlichen Netzwerkzugriff. Private DNS-Zonen unterstützen die interne Auflösung.
Blob Storage	Private Endpoint und privater Container. Der öffentliche Endpunkt bleibt für die Terraform-Verwaltung aktiv. Anonymer Zugriff ist deaktiviert.

Die Weboberfläche ist öffentlich und hat noch keine Benutzeranmeldung. Ein Einsatz mit vertraulichen Dateien benötigt Zugriffsschutz und Aufbewahrungsregeln.

Build und Deployment

Bereitstellung	Das Bash-Skript erstellt die Registry und baut das Image remote in Azure. Ein lokaler Docker-Daemon ist nicht erforderlich.
Terraform	Workspace azure-depot-pdf mit geprüftem Subscription-Bezug. Ein vollständiger Plan folgt auf den Image-Build.
Betrieb	Consumption-Profil, 0,5 vCPU und 1 GiB RAM. Single Revision mit genau einer Instanz für die Streamlit-Sitzung.

Das Skript bestätigt Apply automatisch. State enthält Zugangsdaten und bleibt außerhalb von Git. Für Teamarbeit ist ein geschütztes Remote-Backend der nächste Schritt.

Geprüfter Projektstand

Webanwendung	Startseite erreichbar. Streamlit-Healthcheck antwortet mit HTTP 200 und dem Status ok.
Funktionstest	Beispiel-CSV: 8 Positionen, Gesamtwert 16.700,00 und größtes Positionsgewicht 25,1 %. Tabellen erscheinen vollständig.
Blob-Speicherung	Der Browser bestätigt die Speicherung der hochgeladenen Beispieldatei im privaten Upload-Container.
Infrastruktur	Terraform-Abgleich nach dem Deployment: keine offenen Änderungen. GitHub Actions prüft Build und CLI, ohne Azure automatisch zu deployen.

Grenzen und nächste Schritte

Grenzen des Prototyps

PDFs benötigen erkennbare Tabellen. OCR ist noch nicht implementiert.

Keine Währungsumrechnung. Gemeinsame Summen setzen eine einheitliche Währung voraus.

Klassifizierung und Konzentrationshinweise beruhen auf festen Regeln.

Nächste Schritte

Benutzeranmeldung und Berechtigungen für vertrauliche Uploads ergänzen.

Weitere Bankformate testen und Remote-State einrichten.

Worker, Datenbankintegration und Anwendungstraces ausbauen.

Repository und Live-Demo

GitHub: DepotPDFAzure

Example Azure Depot PDF öffnen